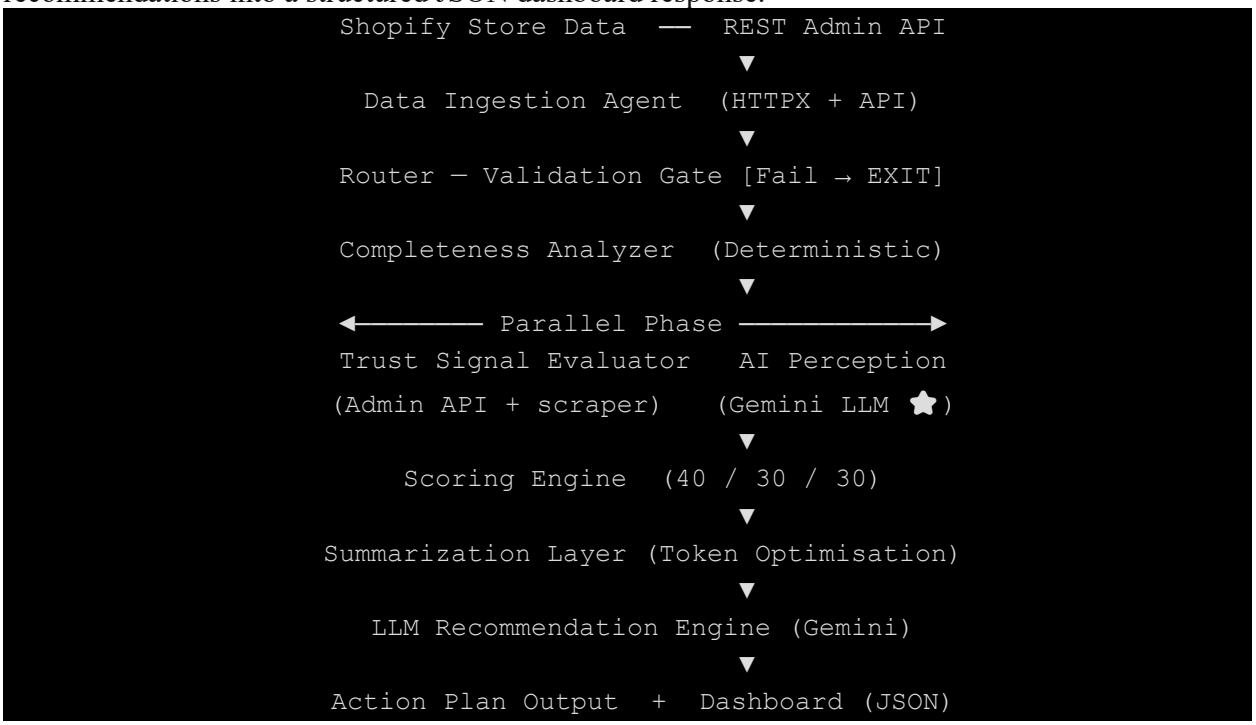
KASPARRO HACKATHON 2026 · STARX

StoreSense-AI

Technical Architecture & Engineering Document

1. System Architecture

StoreSense-AI is a stateless, agentic pipeline orchestrated by LangGraph. Seven specialised agents share a single StoreAnalysisState through a linear-parallel hybrid flow — ingesting Shopify store data, analysing it deterministically, running Trust and Perception checks in parallel, then synthesising LLM-powered recommendations into a structured JSON dashboard response.



Integration	Used By	Purpose
Shopify REST Admin API	Ingestion + Trust Agent	Fetch first 50 products & legal policy pages
Google Gemini Flash Lite	Perception + Recommendation	Buyer trust simulation & natural-language fixes

The pipeline is fully stateless per request — no database, no persistent state. This maximises scalability and simplifies deployment. Client-side localStorage handles any scan history the user wishes to retain.

2. Key Implementation Decisions

Each decision below had a specific engineering reason — not just convenience. We document the why so the tradeoffs are visible.

2.1 LangGraph for Orchestration

WHY	Simple async chains cannot express conditional short-circuiting or true parallel fan-out. LangGraph's StateGraph + conditional edges exit immediately on ingestion failure and run Trust and Perception agents concurrently — critical for both reliability and latency. Implemented in: app/pipeline/graph.py using StateGraph and add_conditional_edges.

2.2 Hybrid Policy Detection (API + Scraper Fallback)

WHY	The Shopify Admin API works even on password-protected stores, but some merchants restrict API scopes. A pure-scraper approach fails on protected storefronts. The hybrid model (API first, keyword-based HTML scraper as fallback) guarantees maximum policy coverage. Crucially, policies that are unreachable due to network errors are marked 'unknown' — not 'missing' — preventing the store from being penalised unfairly. Implemented in: app/agents/trust.py

2.3 Summarization Layer Before LLM

WHY	Passing raw pipeline state — 50 products × multiple fields plus all policy data — directly to the recommendation LLM causes token bloat, increases cost, and dilutes the prompt with low-signal data. The Summarizer condenses this into a focused Store Brief, reducing average prompt size by approximately 60% and ensuring the LLM reasons about strategy rather than parsing raw data. Implemented in: app/agents/summarizer.py

2.4 Stateless Design — No Server Database

WHY	A database layer adds infrastructure overhead, a deployment dependency, and a single point of failure without adding meaningful value at this scale. The system is horizontally scalable and trivially restartable. Persistent scan history is delegated entirely to the client via localStorage. There are no database models or migrations in the codebase — state is initialised per request and discarded after the response is sent.

2.5 Perception Objections as Measurable Score Penalties

WHY	Raw LLM sentiment ('this store feels untrustworthy') is subjective and not actionable. By extracting discrete objection strings from the Perception agent's output and mapping each to a graduated point deduction — first objection −8 pts, second −7 pts, capped at −30 total — we transform qualitative AI judgment into a repeatable, auditable metric that can be debugged and compared across scans. Implemented in: app/agents/scoring.py

3. AI vs. Deterministic Boundary

One of the most deliberate design decisions in StoreSense-AI is exactly where probabilistic AI reasoning ends and deterministic rule-based logic begins. The guiding principle: use deterministic code wherever the answer is a verifiable fact; use AI only where semantic judgment or creative synthesis is required.

Component	Logic Type	Rationale
Ingestion & Completeness	Deterministic	Missing images or empty tag fields are binary facts. An LLM adds cost and uncertainty where none is needed.
Trust Signal Detection	Deterministic	Policy existence is verified via API status codes or keyword matching — consistent, auditable, and fast.
Scoring Engine	Deterministic	Weighted math (40 / 30 / 30) produces scores that are fully explainable and repeatable for identical inputs.
Perception Simulation	AI — LLM	Simulating a human buyer's trust instinct requires the latent world knowledge that hand-written rule-sets cannot encode.
Recommendations	AI — LLM	Synthesising Completeness, Trust, and Perception findings into prioritised, natural-language action items requires creative reasoning.

KEY INSIGHT The deterministic layer sets the scoring floor. A store with a missing return policy will always score low on Trust — the LLM cannot compensate for that real gap by generating persuasive product copy. AI enriches the analysis; it cannot override the facts.

4. Scoring Engine

The final score is a weighted average of three sub-scores, each calculated deterministically from the data collected by the pipeline agents.

```
Final Score = (Completeness × 0.40) + (Trust × 0.30) + (Perception × 0.30)
```

Sub-score	Weight	Formula & Key Penalty Signals
Completeness	40%	(1 − Total Penalty / Max Penalty) × 100. Penalties: missing description −3, missing images −3, missing tags −2, thin title −1, no variants −1.
Trust	30%	(Earned Points / 34) × 100. Signals: Return Policy 10 pts, Shipping 5, Privacy 5, Terms of Service 3, Reviews enabled 5, Pricing consistency 6.
Perception	30%	Confidence base (HIGH = 90, MEDIUM = 65, LOW = 45) minus graduated objection penalties (−8, −7 … capped at −30, floor at 30).

The 40 / 30 / 30 weighting intentionally prioritises data completeness. A store with excellent AI perception but missing product descriptions cannot score above 78 overall — foundational data quality is the prerequisite for trust.

5. Failure Handling

StoreSense-AI follows a 'Never Crash' philosophy. Every failure mode has an explicit fallback path. The system degrades gracefully — always returning a usable response — rather than surfacing an unhandled 500 error.

Failure Mode	Detection Point	Fallback Strategy	User Experience
Shopify 404 / 401 / Timeout	ShopifyFetchError in Ingestion Agent	_route_after_ingest conditional edge short-circuits graph to END immediately	Clear error message: Store not found or inaccessible. No broken dashboard.
LLM 429 — Rate Limit	HTTP 429 response from Gemini API	Retry loop ×3 with exponential back-off; then attempt secondary Gemini model	Minor latency increase only. Output quality unchanged if secondary model succeeds.
LLM Total Failure	All retries and fallback model exhausted	_rule_based_recommendations fires using Completeness + Trust findings	Template-driven action items displayed. Response includes fallback_used: true flag.
Perception Agent Crash	Exception caught inside LangGraph node	Perception sub-score defaults to LOW confidence base (45 pts, zero objections)	Overall score still computed. Status marked partial_success. Notice shown in UI.
Policy URL Unreachable	Network timeout during scraper fallback	Policy recorded as 'unknown' — not 'missing'	Trust score unaffected by network issues. Unknown flag surfaced in dashboard.
Malformed / Invalid Input	Pydantic @field_validator on AnalyzeRequest	422 Unprocessable Entity returned immediately at API boundary	Field-level validation error returned. Zero Shopify API calls or LLM tokens consumed.

System Degradation Principles

Four principles guide every failure and degradation decision in the system:

- Unknown ≠ Missing — stores are never penalised for temporary network conditions. A policy URL that times out is marked unknown and incurs no Trust score penalty.
- Partial results are better than no results — partial_success status surfaces what data is available rather than suppressing the entire analysis when one agent fails.
- Transparency over silence — the fallback_used flag and error arrays are forwarded to the frontend so the dashboard can clearly communicate exactly which insights may be missing or approximated.
- Input sanitisation at the boundary — all URL cleaning and validation happens in the Pydantic model before any agent or LLM prompt is invoked, ensuring no raw user input propagates into the pipeline.

6. Known Limitations

We document these honestly. Each limitation is a deliberate trade-off made under hackathon time constraints, not an oversight.

Limitation	Real-World Impact	Root Cause / Trade-off
Only first 50 products sampled	Completeness score may not represent stores with hundreds of SKUs	Shopify API pagination not implemented; added complexity vs. time available
No server-side scan history	No trend analysis or improvement tracking across multiple scans	Stateless design chosen deliberately for scalability and deployment simplicity
Sequential LLM calls (Perception → Recommendation)	Hard latency floor of 3–7 seconds per analysis that cannot be reduced without architecture changes	Recommendation prompt requires Perception output as a direct input
Policy content not read semantically	Thin or legally vague policies still receive partial Trust credit based on length heuristic alone	Only content-length heuristic used; LLM comprehension of policy text not yet implemented
Single buyer persona in Perception	Trust profile reflects one simulated buyer perspective only	Single Gemini call per analysis to control latency and API cost

7. What We Would Build With More Time

These are the highest-value improvements we identified during development, ranked by impact-to-effort ratio:

Improvement	Expected Impact	Effort
Redis caching with 1-hour TTL on analysis results	Approximately 70% reduction in Shopify API and LLM costs for repeat scans of the same store	Low
Multi-persona Perception — Gift Buyer, Technical Buyer, Bargain Hunter	Broader trust profile from multiple AI viewpoints; richer, more targeted recommendations per audience	Medium
LLM semantic analysis of actual policy content	Detect weak, vague, or contradictory policy language rather than relying on a length heuristic	Medium
Full catalogue sampling via paginated Shopify API calls	Representative Completeness scores for large stores with hundreds or thousands of products	Low
Streaming API responses to the frontend	Perceived response latency drops to approximately 1 second as recommendations stream progressively	Medium
Lightweight server-side scan history store	Enable trend charts, improvement tracking, and before-vs-after comparisons across scans	High